



Análise e Projeto de Sistemas

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Nome: Vitória Freitas

RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2025

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Introdução à Análise e Projeto de Sistemas	Relatório 1
--	---	--------------------

1. Resumo Teórico

O ciclo de vida do desenvolvimento é um conceito que envolve toda a parte da construção de um sistema, desde sua concepção até sua desativação. A compreensão deste ciclo é extremamente importante para eficiência e qualidade do software, possibilitando manutenção durante ele. O ciclo aborda todas as fases de vida do software, como: fase de planejamento, fase de desenvolvimento, fases de testes, fase de implantação e manutenção, por fim, a fase de desativação.

A metodologia ágil é uma metodologia de desenvolvimento de projetos, ela divide os projetos maiores em pequenas partes valorizando a flexibilidade e colaboração. O scrum é uma metodologia ágil que ajuda a equipe a entregar o produto de maneira incremental e colaborativa. O XP é uma metodologia ágil para equipes pequenas a médio tamanho, para desenvolvimento de software em que os requisitos podem mudar rapidamente.

A metodologia em cascata existe desde 1970, surgiu para solucionar problemas de grandes projetos de desenvolvimento de software. Dentro dessa metodologia os projetos são divididos em etapas que se sucedem umas das outras, seguindo um caminho descendente em um formato de cascata. Os processos dentro dessa metodologia incluem: coleta de requisitos, design do produto, implementação, testes e manutenção.

Para exemplificar, vou dar de exemplo a construção de um sistema web. Para o desenvolvimento de um sistema web utilizando uma metodologia ágil você poderia ter uma maior flexibilidade para mudanças de requisitos e se encaixaria melhor em uma equipe de médio a pequeno tamanho. Para desenvolvimento de grande sistema web com uma grande equipe a metodologia cascata, poderia se encaixar bem dentro do desenvolvimento de software.

2. Estudo de Caso

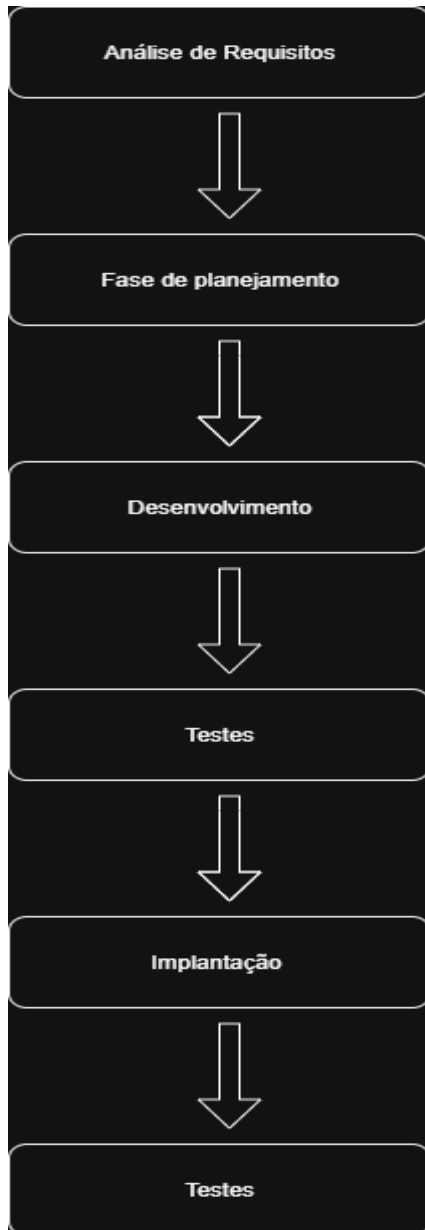
Um sistema de entrega para pedidos de produtos de um supermercado. Neste sistema, clientes podem pedir produtos que estejam disponíveis para serem entregues no local de entrega cadastrado dentro do aplicativo. Desenvolvido pensando na segurança de dados, com criptografia para os dados sensíveis.

As fases incluídas dentro do desenvolvimento desse software foram a análise de

requisitos, a fase de planejamento, desenvolvimento, testes, implantação e manutenção.

Pontos de benefícios das metodologias ágeis seriam a flexibilidade que elas trazem, caso ocorra alguma mudança durante o desenvolvimento do software elas ajudariam a equipe a se adaptar para essas mudanças. E podemos citar, a melhora na colaboração dentro das equipes ao utilizar uma metodologia ágil.

3. Diagrama do Ciclo de Vida



4. Reflexões Finais

Dificuldades para identificar o ciclo de vida, foram para saber quais ciclos são envolvidos dentro do desenvolvimento do software.

Para sabermos qual vai ser ser mais eficiente entre a metodologia ágil e a cascata vai depender do tipo de sistema você e sua equipe estará desenvolvendo, no tamanho dele, sua complexidade, tamanho e experiência de sua equipe e outros fatores.

Aprendi mais sobre como acontece o desenvolvimento de software na prática, sobre

metodologias que são utilizadas e sobre sua eficiência para o desenvolvimento e na melhora na colaboração da equipe.

5. Referências

<https://uds.com.br/blog/ciclo-de-vida-do-software-web/>

<https://monday.com/blog/pt/desenvolvimento/metodologia-agil-x-em-cascata-qual-tipo-de-gere-n-te-voce-e/>

<https://www.scrum.org/resources/what-scrum-module>

<https://www.devmedia.com.br/integrando-xp-as-principais-metodologias-ageis/30989>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Levantamento de Requisitos	Relatório 2
--	---	--------------------

1. Resumo Teórico

Antes de iniciar o desenvolvimento de um software é importante entender as necessidades do cliente, para isso podemos fazer o levantamento de requisitos. Os requisitos fazem a ponte entre o cliente e o programador. Suas principais características são: buscar informações do cliente para construção do software, organizar o projeto e entregar um software de qualidade.

Requisitos funcionais descrevem as ações que o software deve fazer, são as operações que o software deve executar. Por exemplo, o site deve fazer uma busca e filtragem dos dados. Requisitos não funcionais estão ligados a parte da performance do software, como por exemplo: desempenho, segurança, confiabilidade, usabilidade, manutenibilidade e portabilidade.

Entre as principais técnicas de elicitação de requisitos, podemos citar a entrevista, brainstorming e a prototipação. A entrevista é uma reunião com as pessoas envolvidas no projeto (stakeholder) sobre o sistema que será desenvolvido, com ela podemos fazer a elicitação de requisitos e entender as necessidades do cliente. O brainstorming é uma reunião ou várias para que as pessoas possam sugerir e explorar ideias. A prototipação inicia pelo estudo dos requisitos do usuário e um protótipo do sistema para ser validado pelo cliente, os requisitos iniciais servem para fazer um protótipo da interface do usuário, de maneira mais simplificada.

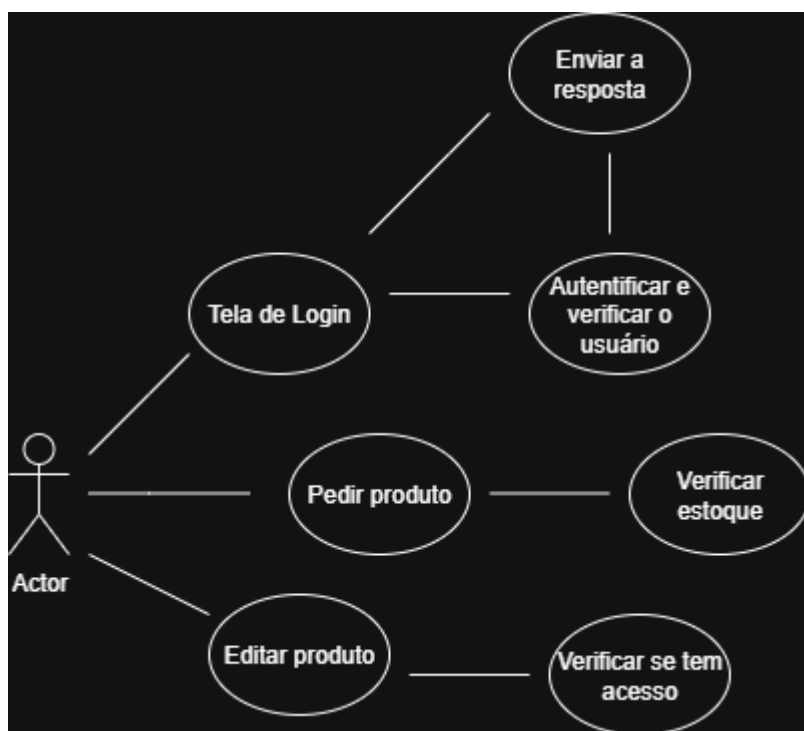
2. Lista de Requisitos

Número do requisitos	Requisitos Funcionais
RF01	O sistema deve permitir login de usuários
RF02	O sistema deve mostrar a quantidade de produtos no estoque.
RF03	O sistema deve ter edição de produtos no estoque.
RF04	O sistema deve ter adição de produtos no estoque.

RF05	O sistema deve ter uma área para usuários administradores e outra para os usuários que vão comprar os produtos.
-------------	---

Número do requisitos	Requisitos Não Funcionais
RNF01	O sistema deve responder em até 2 segundos.
RNF02	O sistema deve ter autenticação.
RNF03	O sistema deve ter criptografia.
RNF04	O sistema deve ter uma interface intuitiva.
RNF05	O sistema deve ter um firewall para proteção contra acessos não autorizados.

3. Diagrama de Caso de Uso



4. Reflexões Finais

Na minha perspectiva as maiores dificuldades em levantar requisitos é entender as necessidades do usuário e saber se ela é funcional ou não funcional. Na minha opinião, a técnica mais eficiente é a de entrevista, pois com ela podemos saber quais são as expectativas e necessidades do cliente. Com essa aula, aprendi melhor sobre o processo de desenvolvimento de software.

5. Referências

<https://www.devmedia.com.br/levantamento-de-requisitos/>

<https://querobolsa.com.br/revista/requisitos-funcionais-e-nao-funcionais>

<https://acertbr.com.br/tecnicas-de-elicitacao-de-requisitos/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Modelagem de Sistemas com UML	Relatório 3
--	--	--------------------

1. Resumo Teórico

A Linguagem de Modelagem Unificada (UML), é uma linguagem gráfica padronizada que tem a função de descrever, projetar e visualizar para documentar sistemas orientados a objetos. Ela é importante, pois auxilia os desenvolvedores com visualização, compreensão, projeção do sistema, identificar padrões, documentação e na detecção de problemas. Ela facilita a comunicação durante o desenvolvimento de software.

2. Sistema Modelado

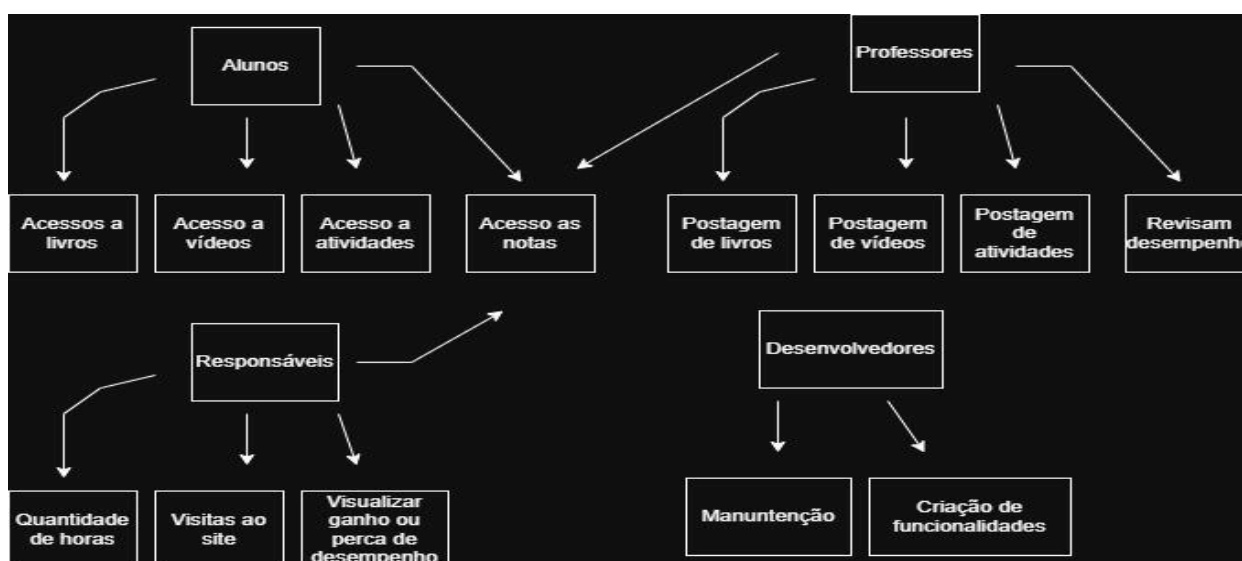
Uma plataforma de ensino educacional para crianças e adolescentes, com o objetivo de conectar alunos, professores e responsáveis.

Atores do sistema:

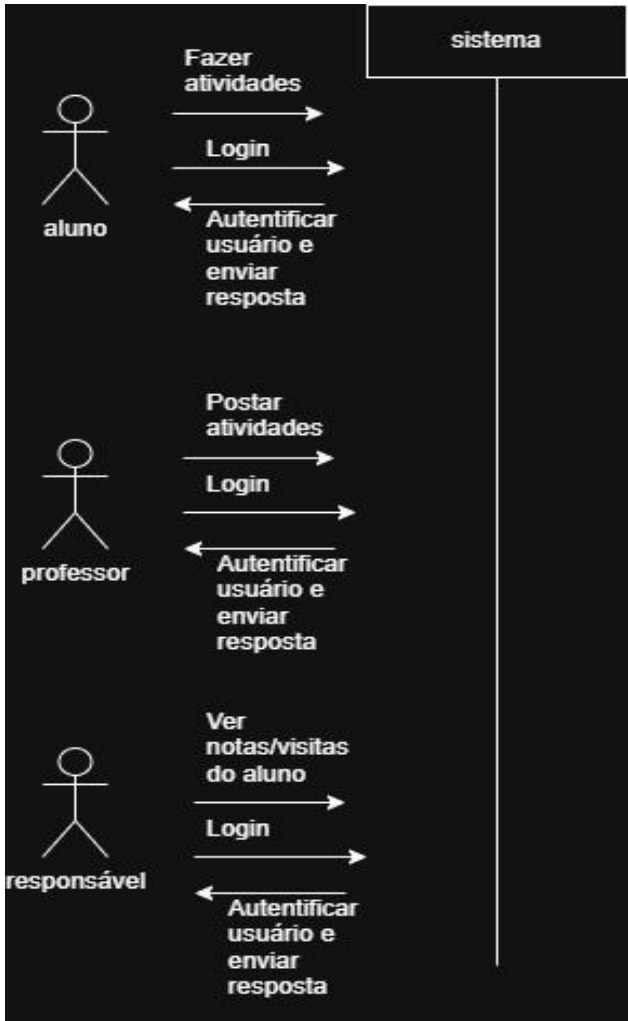
1. Professores: postam atividades, revisam desempenho dos alunos, postam vídeo aulas e livros complementares.
2. Alunos: têm acessos às atividades e livros, aulas complementares.
3. Responsáveis: têm acesso às notas dos alunos, visitas que fazem ao site, quantidades de horas e podem visualizar, se houve melhora ou piora no desempenho do aluno.
4. Desenvolvedores: responsáveis pela criação e manutenção do site.

3. Diagramas Criados

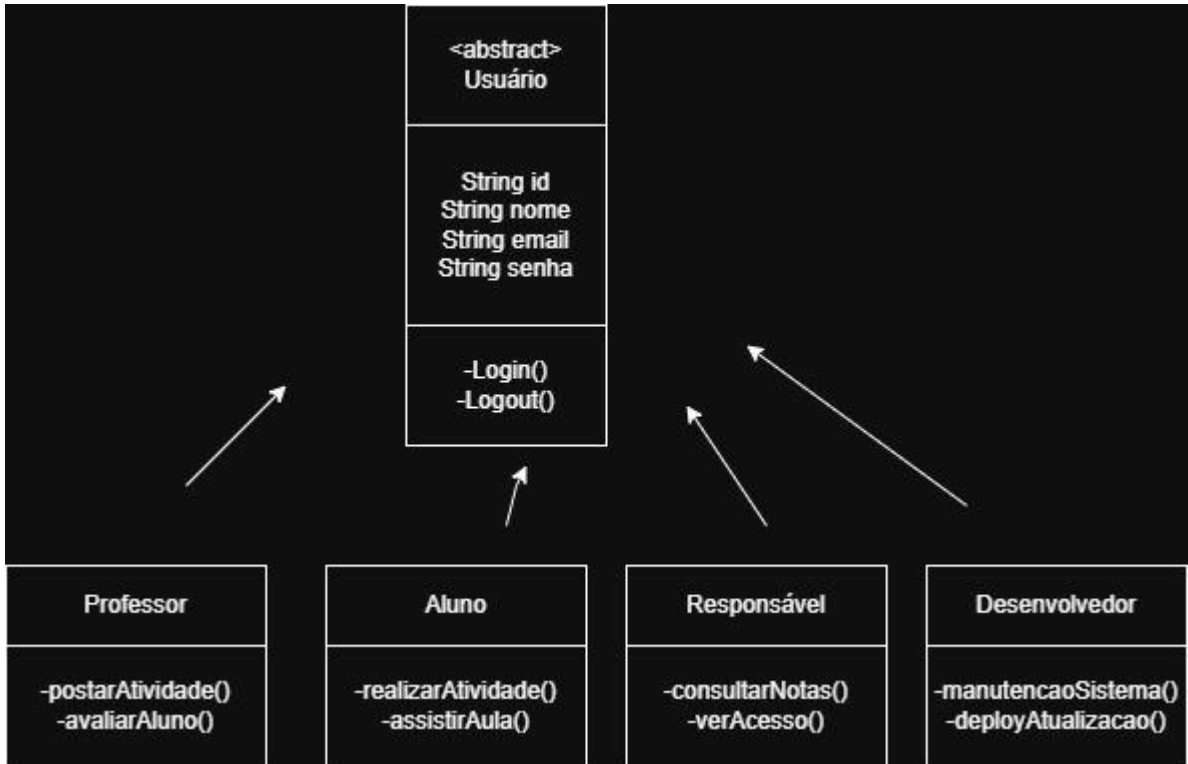
Caso de uso:



Sequência:



Classes:



4. Reflexões Finais

Dificuldade em relação a criação dos diagramas foi entender qual é a proposta de cada um e como encaixar com as necessidades do sistemas. Os diagramas são importantes, pois com eles temos um melhor entendimento sobre o sistema que temos que criar e como ele deve funcionar. Aprendemos na prática como devemos fazer esses diagramas.

5. Referências

<https://www.dio.me/articles/a-importancia-da-uml-na-engenharia-de-software-simplificando-a-complexidade-do-desenvolvimento>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Design de Software	Relatório 4
--	---	--------------------

1. Resumo Teórico

Padrões de design são soluções genéricas para problemas comuns nos códigos de desenvolvimento do software. Eles são um conceito geral para ajudar a resolver problemas recorrentes. São usados, porque facilitam o trabalho dos desenvolvedores e aumentam a produtividade, permitindo a eles criar códigos melhores e de maneira mais rápida. Exemplos de Padrões de design: Singleton, classes tem apenas uma instância; Factory, cria objetos em especificar a classe; Observer, define dependência para muitos objetos. A arquitetura multicamadas tem um princípio básico que o cliente não se comunica diretamente com o servidor de banco de dados e sim intermediária (como o banco de dados), essa arquitetura utiliza objetos distribuídos e interfaces para realizar suas operações. A arquitetura multicamadas e os padrões de design são elementos complementares, que deixam o desenvolvimento mais eficiente.

2. Código Desenvolvido

Código Python usando o padrão de design Singleton:

```
class ConnectionManager:
    _instance = None
    _connections = {}

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            print("ConnectionManager criado pela primeira vez")
        return cls._instance

    def __init__(self):
        # O __init__ será chamado toda vez, mas a instância é a mesma
        print("ConnectionManager inicializado")

    def add_connection(self, name, connection):
        self._connections[name] = connection
```

```

        print(f"Conexão '{name}' adicionada")

    def get_connection(self, name):
        return self._connections.get(name)

    def close_all(self):
        for name in self._connections:
            print(f"Fechando conexão: {name}")
        self._connections.clear()

manager1 = ConnectionManager()
manager1.add_connection("database", "mysql_connection")

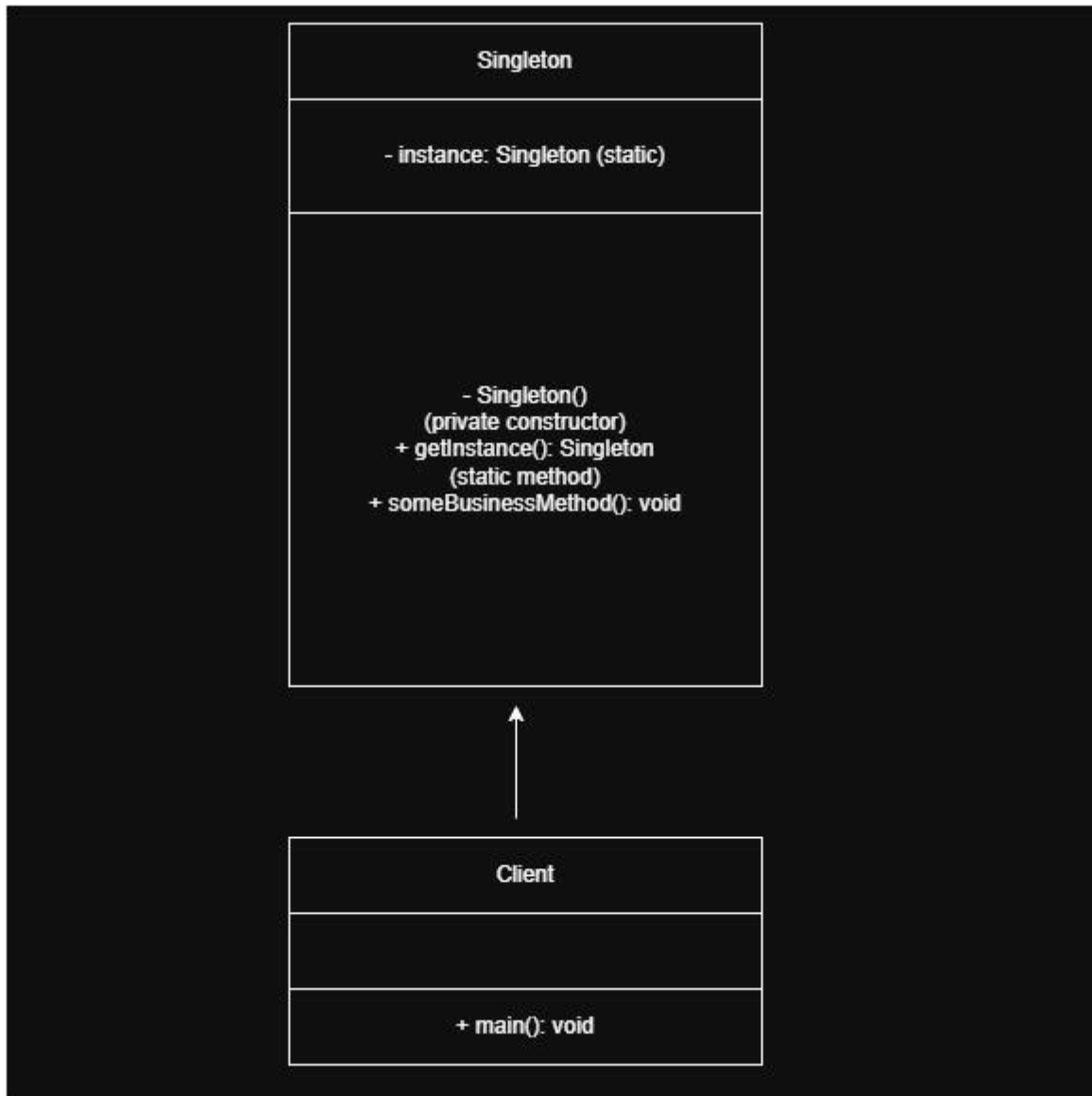
# manager2 NÃO é uma nova instância, é apenas outra referência à MESMA
# instância
manager2 = ConnectionManager()
manager2.add_connection("api", "rest_api_connection")

print(f"manager1 é manager2? {manager1 is manager2}") # True
print(f"Conexões no manager1: {manager1._connections}")
print(f"Conexões no manager2: {manager2._connections}")

# Todas as referências apontam para o mesmo objeto
manager3 = ConnectionManager()
print(f"Conexões no manager3: {manager3._connections}")

```

3. Diagrama UML



4. Reflexões Finais

Dificuldades para implementar foi entender como cada padrão de design funciona e como implementá-las dentro do código. Esses padrões ajudam a solucionar problemas recorrentes dentro dos códigos. Com essa aula aprendi na prática como implementar esses padrões dentro de projetos.

5. Referências

<https://uds.com.br/blog/a-importancia-dos-design-patterns-no-dev-de-software/>

<https://www.devmedia.com.br/introducao-ao-modelo-multicamadas/5541>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Arquitetura de Software	Relatório 5
--	--	--------------------

1. Resumo Teórico

A arquitetura de software é o que define a estrutura e os padrões de um software. O arquiteto de software é o responsável pela estruturação do software, irá definir padrões técnicos que devem ser seguidos durante o desenvolvimento do sistema e isso inclui a arquitetura do software. A escolha da arquitetura de software impacta na performance do sistema, de maneira positiva ou negativa, ela tem que suportar as mudanças que podem ocorrer durante o desenvolvimento.

Arquitetura de microsserviços é uma coleção de pequenos serviços independentes, cada serviço é um código diferente e cada um implementa uma funcionalidade, se comunicando através de APIs.

Na Service-Oriented Architecture (SOA) os componentes são reutilizáveis e também são disponibilizados na forma de serviço, isso significa que cada um tem funcionalidades de software independentes.

A arquitetura monolítica é um modelo em que uma base de código pode executar diversas funções de negócio. Nela todo código necessário para um aplicação é mantido em um local central, funcionando com um único arquivo executável ou diretório.

A arquitetura que irá ser escolhida é a base que determina como o sistema irá se comportar em relação ao seu crescimento (escalabilidade) e quando for modificado (manutenção), por exemplo em na arquitetura monolítica a escalabilidade é prejudicada, já na de microsserviços é facilitada.

2. Sistema Modelado

Sistema escolhido: FastDelivery, uma plataforma para delivery de comidas. Criada com o objetivo de conectar o restaurante ao clientes, de maneira rápida e intuitiva. Os usuários podem pedir suas comidas, ver o cardápio, acompanhar a entrega. O restaurante gerencia seu cardápio e pedidos. Os entregadores recebem notas.

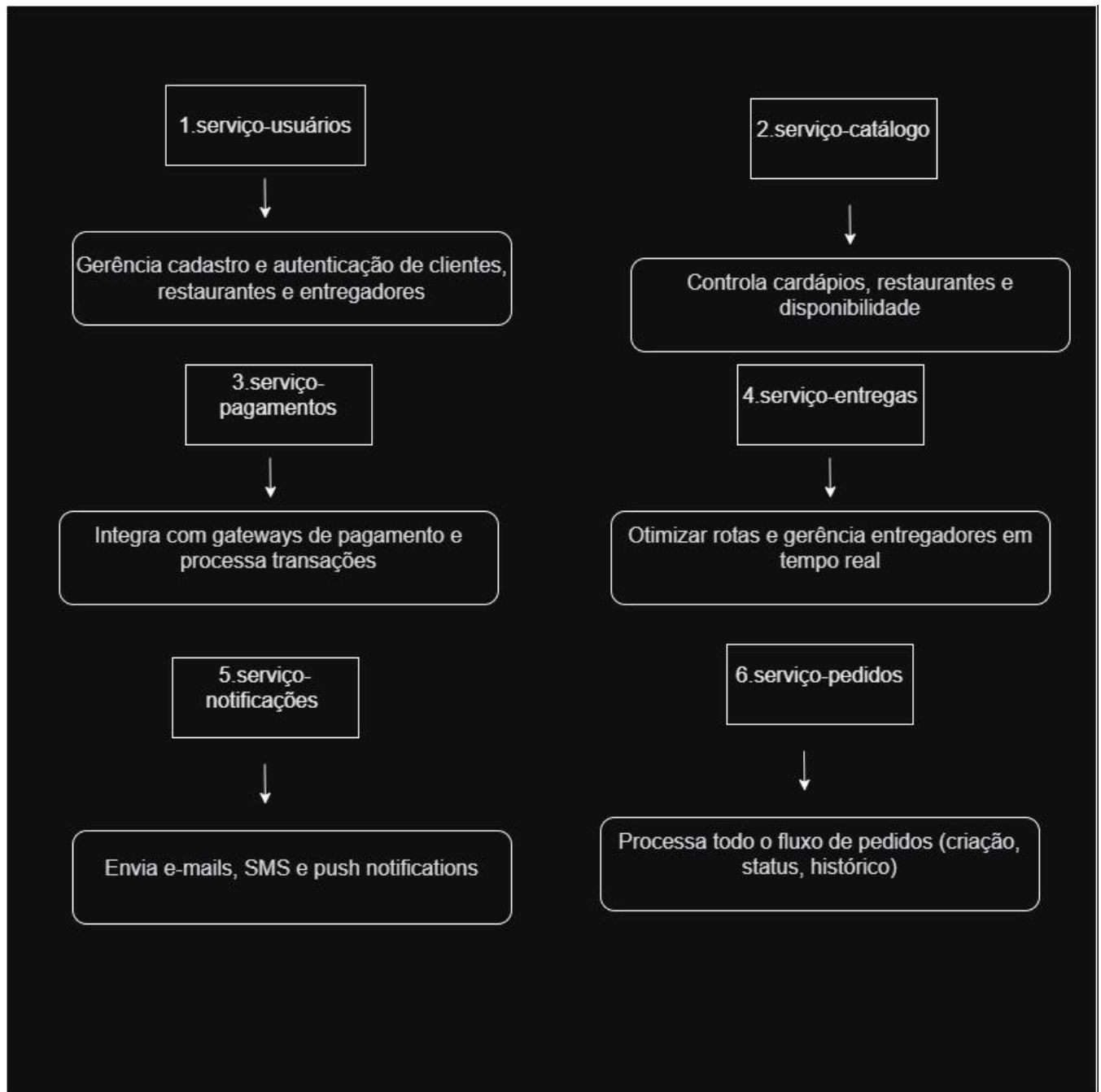
Levando em consideração esse cenário, a arquitetura microsserviços é a escolha ideal, pois favorece a escalabilidade e manutenção do sistema.

Serviços propostos:

1.serviço-usuários	Gerência cadastro e autenticação de
--------------------	-------------------------------------

	clientes, restaurantes e entregadores
2.serviço-catálogo	Controla cardápios, restaurantes e disponibilidade
3.serviço-pagamentos	Integra com gateways de pagamento e processa transações
4.serviço-entregas	Otimizar rotas e gerência entregadores em tempo real
5.serviço-notificações	Envia e-mails, SMS e push notifications
6.serviço-pedidos	Processa todo o fluxo de pedidos (criação, status, histórico)

3. Diagramas Criados



4. Reflexões Finais

As dificuldades na modelagem da arquitetura é escolher a arquitetura ideal para aquele sistema específico que irá desenvolver, levando em consideração suas necessidades e o contexto do desenvolvimento.

Para o sistema que criamos, acredito que a melhor arquitetura seria a de microserviços, pois com ela a escalabilidade e manutenção são facilitadas.

Na prática aprendemos mais como a arquitetura de um software funciona e qual é o seu impacto no desenvolvimento de um sistema.

5. Referências

<https://uds.com.br/blog/arquitetura-de-software-o-que-e/>

https://www-ibm-com.translate.goog/think/topics/monolithic-architecture?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Ferramentas de Modelagem e Case	Relatório 6
--	--	--------------------

1. Resumo Teórico

Ferramentas CASE (Computer Aided Software Engineering), são ferramentas que auxiliam o processo de desenvolvimento de software, como compiladores, editores estruturados, sistemas de controles de código fonte e ferramentas de modelagem. São compostas por software complexos para auxiliar desenvolvedores.

Astah oferece ferramentas de modelagem para criação de UML, diagrama ER, diagramas de fluxo de dados, fluxogramas, mapas mentais. Visual Paradigm é uma ferramenta CASE para UML (Unified Modeling Language). [Draw.io](https://draw.io) é um software para criação de aplicativos de diagramação.

Essas ferramentas facilitam o desenvolvimento de software, porque com elas os desenvolvedores podem entender qual funcionalidades o sistema deve ter, quais são as necessidades do cliente e como a equipe de desenvolvedores podem se organizar.

2. Diagramas Criados, Ferramenta: Draw.io

Diagrama de caso de uso

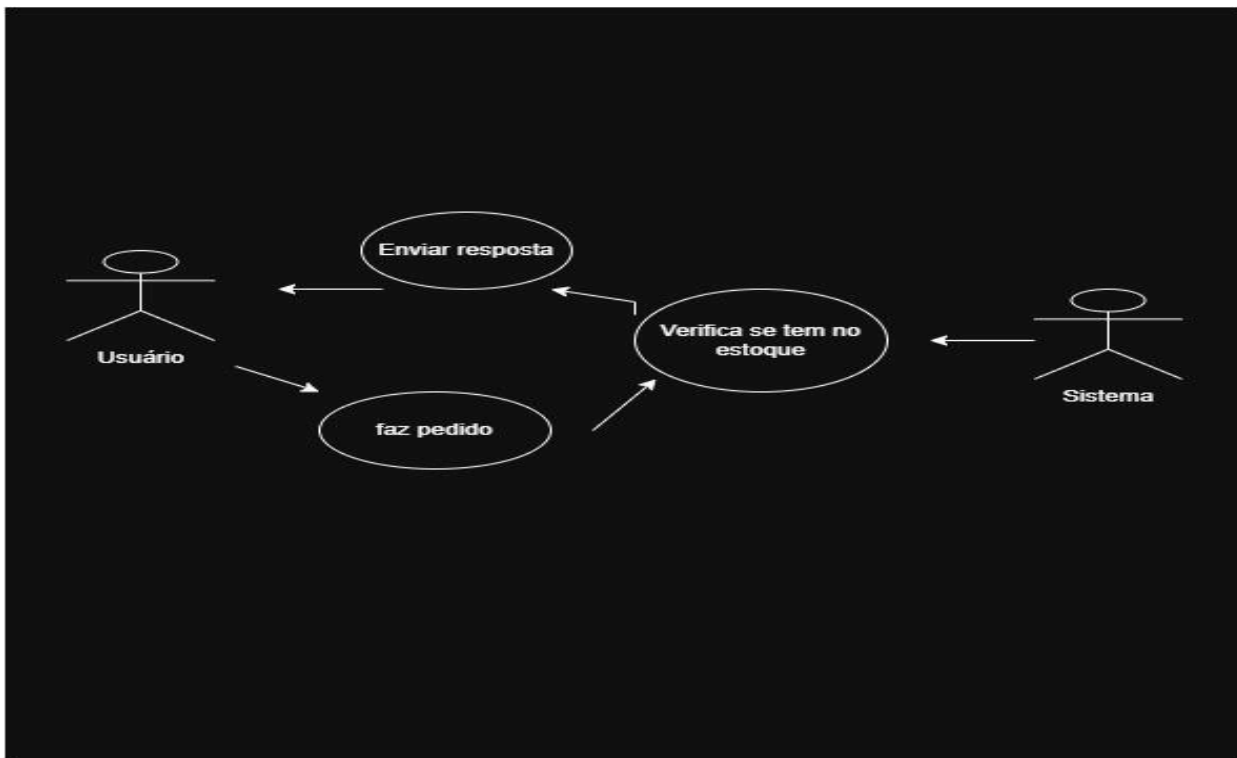


Diagrama de classes

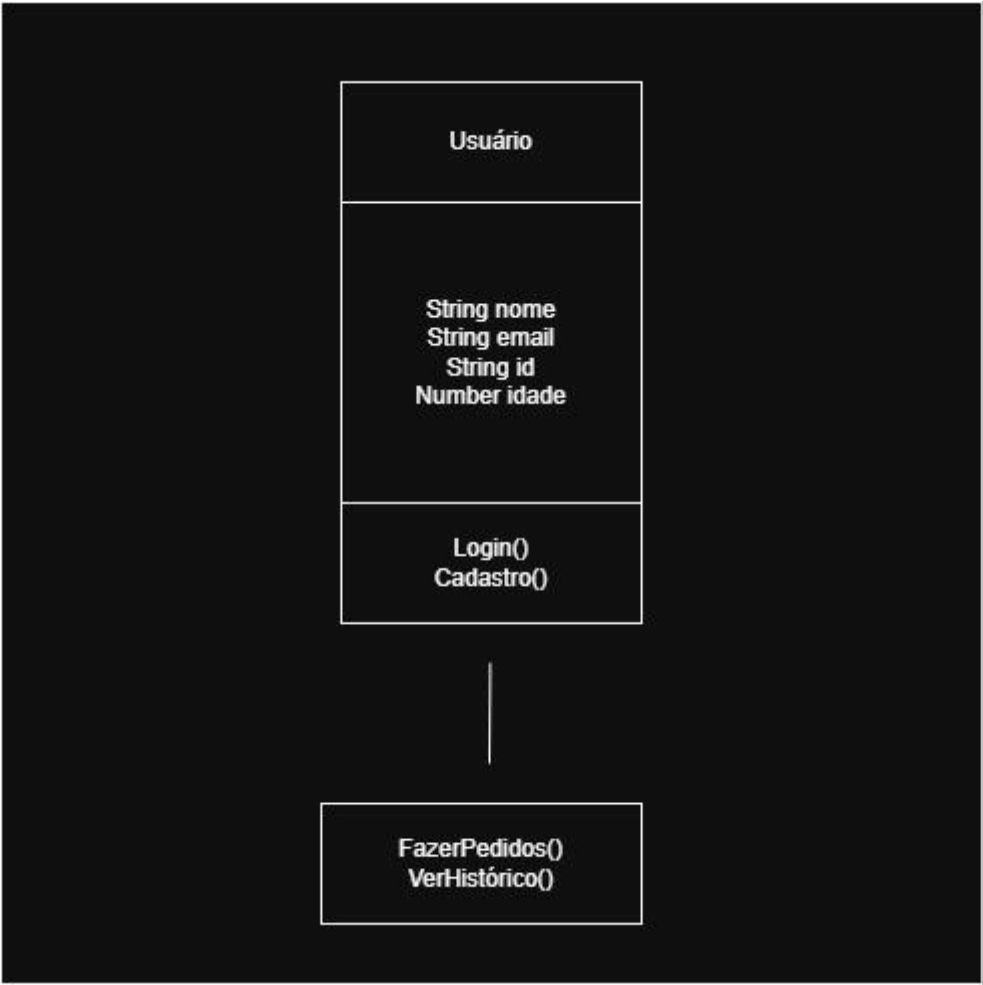
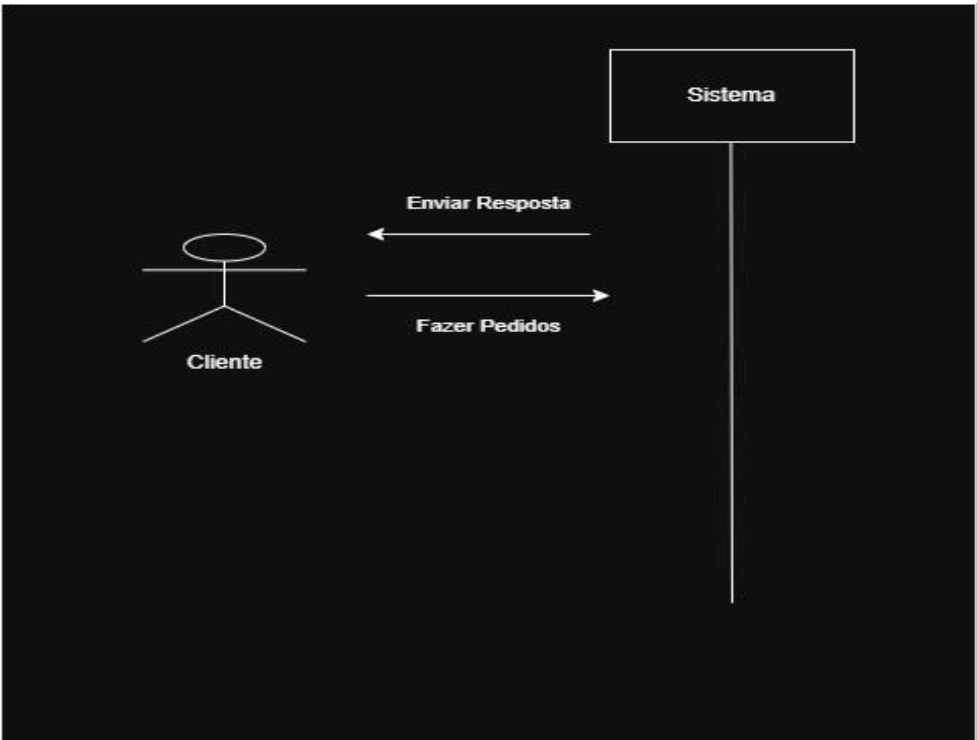


Diagrama de sequência



4. Reflexões Finais

Benefícios na utilização são que elas facilitam o desenvolvimento e são intuitivas de se utilizar, sendo extremamente importantes para ter um desenvolvimento de software mais produtivo e organizado. E com elas podemos documentar nosso software de maneira mais fácil, pois elas já indicam o que nosso sistema irá fazer, quais serão os papéis de cada pessoa envolvida com ele, podem indicar quais ferramentas utilizamos, como por exemplo, linguagens e framework e também operações que ocorrem dentro do código desenvolvido para a plataforma. Na prática aprendemos como utilizar essas ferramentas e sua importância dentro do desenvolvimento.

5. Referências

<https://www.devmedia.com.br/ferramentas-case-e-qualidade-dos-dados-o-paradigma-da-boua-modelagem/6905>

<https://astah.net/pt/>

<https://www.devmedia.com.br/artigo-sql-magazine-42-modelagem-de-dados-com-a-visual-paradigm-do-modelo-de-classes-a-criacao-do-banco-de-dados/7019>

https://www-drawio-com.translate.goog/about?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Avaliação e Validação de Projetos de Sistemas	Relatório 7
--	--	--------------------

1. Resumo Teórico

Com processos de verificação e validação de projetos de sistemas podemos verificar se está tudo certo com o sistema e se ele é o certo, com processo de verificação vemos se o sistema cumpre com os requisitos e as características desejadas e com a validação vemos se o sistema cumpre com os objetivos de negócio do cliente, com isso eles se tornam extremamente importantes para o processo de construção de um software.

O processo de verificação é feito pela identificação de erros, defeitos ou falhas usando métodos, técnicas, padrões ou regras apropriados.

Com o processo de validação vemos se o sistema quando em uso está satisfazendo as necessidades do usuário. O objetivo desse processo é criar confiança que ele cumpre com o que ele deveria fazer, satisfazendo os requisitos de stakeholders.

Exemplos de métodos de revisão colaborativa: Inspeção de Software (Fagan Inspection), é um tipo mais formal de revisão e Walkthrough é um conjunto de revisão com o objetivo de melhorar a qualidade do produto.

2. Checklist de Avaliação

#	Item de Verificação	Sim	Não
1	A documentação possui uma introdução/clara que explica o propósito e o escopo do sistema?	X	☐
2	O glossário de termos técnicos e de negócio está definido e é consistente?	☐	X
3	Os usuários-alvo do sistema estão claramente identificados?	X	☐
4	Os requisitos não-funcionais são especificados?	X	☐

3. Resultados da Avaliação

Com os resultados da avaliação podemos dizer que os problema notados é que os termos de negócio não foram definidos, isso seria a correção necessária a ser feita. Seus pontos fortes são os requisitos não-funcionais bem especificados e a introdução sobre o propósito do sistema foi bem definida.

4. Reflexões Finais

Durante a avaliação percebemos os pontos fracos dos nossos documentos e o que deve ser melhorado, assim aprendendo na prática como devemos avaliar esses documentos para desenvolvimento de sistemas e com feedback melhoramos eles, pois arrumamos o que estava errado e colocamos complementos nele.

5. Referências

<https://ezute.org.br/verificacao-e-validacao-de-sistemas/>

https://www-sciencedirect-com.translate.goog/topics/computer-science/software-inspection?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

<https://sites.google.com/site/alunopragmatico/engenharia-de-software/t%C3%A9cnicas-de-revis%C3%A3o-de-software-inspe%C3%A7%C3%A3o-walkthrough-pair-programming>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Manutenção e Evolução de Sistemas	Relatório 8
--	--	--------------------

1. Resumo Teórico

Manutenção corretiva corrige defeitos no software por meio de atualizações rápidas que acontecem de forma recorrente. Quem for responsável pela manutenção do software deve implementar as correções rapidamente e frequentemente.

Manutenção adaptativa é uma mudança no software para manter ele utilizável depois de uma mudança no seu ambiente operacional. Também ocorre quando acontece uma mudança na infraestrutura de armazenamento em nuvem que impacta a sua funcionalidade.

Manutenção preventiva é a modificação do software para detectar erros e corrigir eles, antes que eles causem problemas, assim prolongando a vida útil do sistema.

Manutenção perfectiva ela serve para refinar e aprimorar funcionalidades e usabilidades do software. Pode também adicionar novos recursos e atualização de componentes.

Refatoração de código é o processo de melhoria da estrutura e legibilidade do código do programa sem acabar alterando o comportamento de suas funcionalidades. Tem o objetivo de deixar o código mais eficiente.

Reengenharia de software é o processo de examinar e modificar um sistema de software para reconstruí-lo, com o objetivo de melhorar suas funcionalidades e seu desempenho.

Um exemplo prático de evolução de um sistema é o Microsoft Windows que teve o seu sistema original em 2001 com o Windows XP, depois em 2009 teve o Windows 7, 2015 veio com o Windows 10 e em 2021 com o Windows 2021. E cada lançamento veio com aprimoramentos e novas funcionalidades para o sistema operacional.

2. Código Original

```
const idade = 18
let aprovado = false

if (idade >= 18) {
  aprovado = true
}
```

```
} else {  
    aprovado = false  
}  
  
if(aprovado) {  
    console.log("Aprovado")  
} else {  
    console.log("Negado")  
}
```

3. Código Refatorado

```
const idade = 18  
let aprovado = idade >= 18 ? true : false  
const mensagem = aprovado ? "Aprovado" : "Negado"  
console.log(mensagem)
```

4. Reflexões Finais

Dificuldades para manutenção é identificar quais são os erros e o que pode ser melhorado dentro do sistema. Nesse código refatorado que foi mostrado acima, se pode notar que as melhoras foram redução de linhas de código, remove a repetição de if/else, fica claro que “aprovado” depende diretamente da idade e a lógica de aprovação e mensagem estão separadas. Assim aprendemos na prática o que refatoramento de código significa e como trabalhar com ele.

5. Referências

https://www.baytechconsulting-com.translate.goog/blog/successful-software-maintenance?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc&_x_tr_hist=true

<https://pt.itpedia.nl/2024/08/03/de-verschillen-tussen-code-refactoring-en-re-engineering-van-sftware/>